Chapter 7
(Downplaying) Pointers in Modern C++

Chapter Targets

- Learn what pointers are.
- Declare and initialize pointers.
- Use the pointer operators & (address) and * (indirection).
- Compare the capabilities of pointers and references.
- Pass arguments to functions by reference using pointers.
- Use pointer-based arrays and strings mostly in legacy code.
- Use const with pointers and the data they point to.
- Determine the number of bytes that store a value of a particular type using sizeof.
- Understand legacy-code pointer expressions and pointer arithmetic.
- Use nullptr to represent pointers to nothing.
- Use functions begin and end with pointer-based arrays.
- Learn C++ Core Guidelines for avoiding pointers and pointer-based arrays to create safer, more robust programs.
- Use C++20's to_array function to convert built-in arrays and initializer lists to std::arrays.

Introduction

- This chapter discusses pointers, built-in pointer-based arrays and pointer-based strings (also called C-strings), each of which C++ inherited from the C programming language.
- **Downplaying** Pointers in Modern C++
  - Pointers are powerful but challenging to work with and error-prone.
  - So, Modern C++ has added features that eliminate the need for most pointers.
  - New software-development projects generally should prefer:
    - using references or “smart pointer” objects (Chapter 11) over pointers,
    - using std::array and std::vector objects (Chapter 6) over built-in pointer-based arrays, and
    - using std::string objects and std::string_view objects (Chapters 2 and 8) over pointer-based C-strings.

Introduction

- Sometimes Pointers Are Still Required
  - You'll frequently encounter pointers, pointer-based arrays (also called C-style arrays) and pointer-based C-strings in the massive installed base of legacy C++ code.
  - Pointers are required to:
    - create and manipulate dynamic data structures, like linked lists, queues, stacks and trees that can grow and shrink at execution time—though most programmers will use the C++ standard library's existing dynamic containers like vector and the containers we discuss in Chapter 13,
    - process command-line arguments, which a program receives as a pointer-based array of C-strings, and
    - pass arguments by reference if there's a possibility of a nullptr—a reference must refer to an actual object.

Introduction

- Pointer-Related C++ Core Guidelines
  - We mention C++ Core Guidelines for avoiding pointers to make your code safer and more robust, pointer-based arrays and pointer-based strings.
  - For example, several guidelines recommend implementing pass-by-reference using references rather than pointers.

Introduction

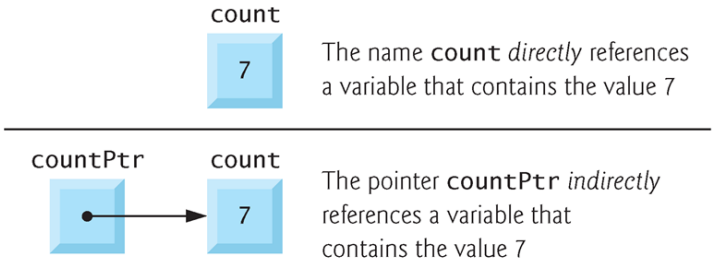
- C++20 Features for Avoiding Pointers
  - For programs that still require pointer-based arrays (e.g., command-line arguments), C++20 adds two new features that help make your programs safer and more robust:
    - Function to_array converts a pointer-based array to a std::array, so you can take advantage of the features we demonstrated in Chapter 6.
    - spans offer a safer way to pass built-in arrays to functions. They're iterable, so you can use them with range-based for statements to conveniently process elements without risking out-of-bounds array accesses. Also, because spans are iterable, you can use them with standard library container-processing algorithms, such as accumulate and sort. In this chapter, you'll see that spans also work with std::array and std::vector.
  - The key takeaway from reading this chapter is to avoid using pointers, pointer-based arrays and pointer-based strings whenever possible.
  - If you must use them, take advantage of to_array and spans.

Introduction.

- Other Concepts Presented in This Chapter
  - We declare and initialize pointers and demonstrate the pointer operators & and *.
    - Here, we show that pointers also enable pass-by-reference. We demonstrate built-in, pointer-based arrays and their intimate relationship with pointers.
    - We show how to use const with pointers and the data they point to.
    - We introduce the sizeof operator to determine the number of bytes that store values of particular fundamental types and pointers.
    - We also demonstrate pointer expressions and pointer arithmetic.
  - C-strings were used widely in older C++ software.
    - This chapter briefly introduces Cstrings.
    - You'll see how to process command-line arguments—a simple task for which C++ still requires C-strings and pointer-based arrays.

Pointer Variable

- Pointers are variables whose values are memory addresses.
  - Normally, a variable directly contains a specific value.
  - A pointer, on the other hand, contains an address of a variable that contains a specific value.
  - In this sense, a variable name directly references a value, and a pointer indirectly references a value.
  - Referencing a value through a pointer is called indirection.



Pointer Variable (Cont.)

- Declaring Pointers
  - Pointers, like all variables, must be defined before they can be used.
  - The definition
    `int *countPtr{nullptr}, count;`
    specifies that variable `countPtr` is of type `int *` (i.e., a pointer to an integer) and is read (right to left), "`countPtr` is a pointer to `int`" or "`countPtr` points to an object of type `int`."
    - The `*` applies only to `countPtr` in the definition.
    - The variable `count` is defined to be an `int`, not a pointer to an int.

Pointer Variable (Cont.)

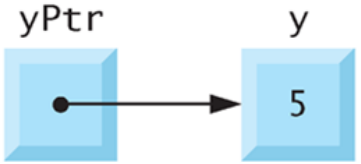
- Pointers can be defined to point to objects of any type.
  - When `*` is used in this manner in a definition, it indicates that the variable being defined is a pointer.
  - To prevent the ambiguity of declaring pointer and non-pointer variables in the same declaration as shown above, you should always declare only one variable per declaration.

Pointer Variable.

- Initializing and Assigning Values to Pointers
  - Initialize each pointer with nullptr or a memory address.
    - A null pointer has the value nullptr and “points to nothing.”
    - From this point forward, when we refer to a “null pointer,” we mean a pointer with the value nullptr.
    - Initialize all pointers to prevent pointing to unknown or uninitialized areas of memory.
  - In legacy C++, null pointers were specified by 0 or NULL.
    - NULL is defined in several standard library headers to represent the value 0.
  - For modern C++, the C++ Core Guidelines recommend always using nullptr because it’s more readable and cannot be confused with an int value.

Pointer Operators

- The `&`, the address operator, is a unary operator that returns the address of its operand.
  - For example, assuming the definitions
    ```
    int y{5};
    int *yPtr{nullptr};
    ```
    the statement
    ```
    yPtr = &y;
    ```
    assigns the address of the variable y to pointer variable `yPtr`.



  - Variable `yPtr` is then said to "point to" `y`.
  - The operand of the address operator must be a variable.
    - The address operator cannot be applied to constants or expressions.

Pointer Operators (Cont.)

- Pointer Representation in Memory
  - Assuming that integer variable `y` is stored at location 600000, and pointer variable `yPtr` is stored at location 500000.

Pointer Operators (Cont.)

- The Indirection (`*`) Operator
  - The unary `*` operator, commonly referred to as the indirection operator or dereferencing operator, returns the value of the object to which its operand (i.e., a pointer) points.
  - For example, the statement
    ```
    printf("%d", *yPtr);
    ```
    prints the value of variable `y`, namely 5.
  - Using `*` in this manner is called dereferencing a pointer.
  - Note that dereferencing values may cause memory-access violation or segmentation-faults if the compiler assumes the values as pointers; otherwise, the compiler will issue an error of mismatch in types.

Pointer Operators (Cont.)

- Using the Address (&) and Indirection (*) Operators
  - The next program demonstrates the & and * pointer operators, which have the third-highest precedence.
  - Memory locations are output by << in this example as hexadecimal (i.e., base-16) integers.

```
1   // Pointer operators & and *.
2   #include <iostream>
3   using namespace std;
4
5   int main() {
6      constexpr int a{7}; // initialize a with 7
7      const int* aPtr{&a}; // initialize aPtr with address of int variable a
8
9      std::cout << "The address of a is " << &a
10        << "\nThe value of aPtr is " << aPtr;
11     std::cout << "\n\nThe value of a is " << a
12        << "\nThe value of *aPtr is " << *aPtr << '\n';
13  }
```

```
The address of a is 000000D649EFFD00
The value of aPtr is 000000D649EFFD00

The value of a is 7
The value of *aPtr is 7
```

Pointer Operators (Cont.)

- Using the Address (&) and Indirection (*) Operators
  - The output shows that variable a's address (line 9) and aPtr's value (line 10) are identical, confirming that a's address was indeed assigned to aPtr (line 7).
  - The outputs from lines 11–12 confirm that *aPtr has the same value as a.

```
1  // Pointer operators & and *.
2  #include <iostream>
3  using namespace std;
4
5  int main() {
6     constexpr int a{7}; // initialize a with 7
7     const int* aPtr{&a}; // initialize aPtr with address of int variable a
8
9     std::cout << "The address of a is " << &a
10       << "\nThe value of aPtr is " << aPtr;
11    std::cout << "\n\nThe value of a is " << a
12       << "\nThe value of *aPtr is " << *aPtr << '\n';
13 }
```

```
The address of a is 000000D649EFFD00
The value of aPtr is 000000D649EFFD00

The value of a is 7
The value of *aPtr is 7
```

Pointer Operators.

- Using the Address (&) and Indirection (*) Operators
  - The displayed memory addresses are compiler- and platform-dependent.
    - They typically change with each program execution, so you'll likely see different addresses.

```
1   // Pointer operators & and *.
2   #include <iostream>
3   using namespace std;
4
5   int main() {
6      constexpr int a{7}; // initialize a with 7
7      const int* aPtr{&a}; // initialize aPtr with address of int variable a
8
9      std::cout << "The address of a is " << &a
10        << "\nThe value of aPtr is " << aPtr;
11     std::cout << "\n\nThe value of a is " << a
12        << "\nThe value of *aPtr is " << *aPtr << '\n';
13  }
```

```
The address of a is 000000D649EFFD00
The value of aPtr is 000000D649EFFD00

The value of a is 7
The value of *aPtr is 7
```

Pass-by-Reference with Pointers

- There are three ways in C++ to pass arguments to a function:
  - pass-by-value,
  - pass-by-reference with a reference argument and
  - pass-by-reference with a pointer argument (sometimes called pass-by-pointer).
- Here, we explain pass-by-reference with a pointer.
  - Pointers, like references, can be used to modify variables in the caller or pass large data objects by reference to avoid the overhead of copying objects.
  - When calling a function that receives a pointer, pass a variable's address by applying the address operator (&) to the variable's name.

Pass-by-Reference with Pointers (Cont.)

```
15  // calculate and return cube
16  int cubeByValue(int n) {
17     return n * n * n; // cube
18  }
```

- An Example of Pass-By-Value
  - The next two program present two functions that cube an integer.
    - The first program passes variable number by value (line 11) to function cubeByValue (lines 16–18), which cubes the local copy of its argument and passes the result back to main using a return statement (line 17).
    - We overwrite and store the new value in number (line 11).

```
1   // Pass-by-value used to cube a variable's value.
2   #include <format>
3   #include <iostream>
4
5   int cubeByValue(int n); // prototype
6
7   int main() {
8      int number{5};
9
10     std::cout << std::format("Original value of number is {}\n", number);
11     number = cubeByValue(number); // pass number by value to cubeByValue
12     std::cout << std::format("New value of number is {}\n", number);
13  }
```

```
Original value of number is 5
New value of number is 125
```

Pass-by-Reference with Pointers (Cont.)

- An Example of Pass-By-Reference with Pointers
  - The second program passes the variable number to cubeByReference using pass-by-reference with a pointer argument (line 12).
    - So, the address of number is passed to the function.

```
1   // Pass-by-reference with a pointer argument used to cube a
2   // variable's value.
3   #include <format>
4   #include <iostream>
5
6   void cubeByReference(int* nPtr); // prototype
7
8   int main() {
9      int number{5};
10
11     std::cout << std::format("Original value of number is {}\n", number);
12     cubeByReference(&number); // pass number address to cubeByReference
13     std::cout << std::format("New value of number is {}\n", number);
14  }
```

```
Original value of number is 5
New value of number is 125
```

Pass-by-Reference with Pointers (Cont.)

- An Example of Pass-By-Reference with Pointers
  - Function cubeByReference (lines 17–19) specifies parameter nPtr (a pointer to int) to receive its argument.
    - The function uses the dereferenced pointer—*nPtr, an alias for number in main—to cube the value to which nPtr points (line 18).
    - This directly changes the value of number in main (line 9).
  - Line 18 can be made clearer with redundant parentheses: *nPtr = (*nPtr) * (*nPtr) * (*nPtr); // cube *nPtr

```
16  // calculate cube of *nPtr; modifies variable number in main
17  void cubeByReference(int* nPtr) {
18     *nPtr = *nPtr * *nPtr * *nPtr; // cube *nPtr
19  }
```

Pass-by-Reference with Pointers (Cont.)

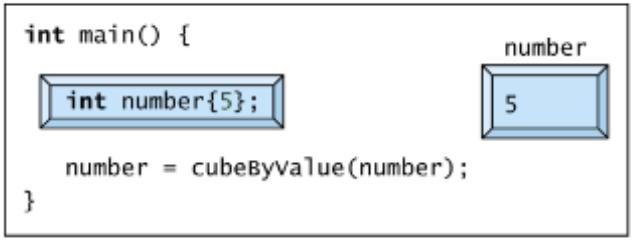
- An Example of Pass-By-Reference with Pointers
  - A function receiving an address as an argument must define a pointer parameter to receive the address.
    - For example, function cubeByReference's header (line 17) specifies that the function receives a pointer to an int as an argument, stores the address in nPtr and does not return a value.
    - It does not need to return a value because line 18 assigns the cubed value directly to number in main.

Pass-by-Reference with Pointers.

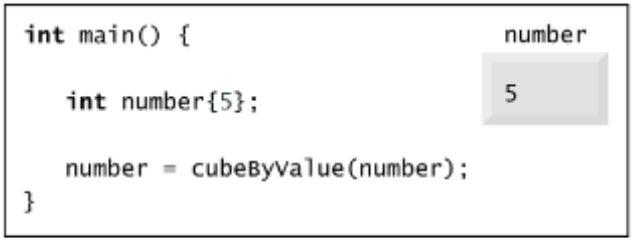
- Insight: Pass-By-Reference with a Pointer Actually Passes the Pointer By Value
  - Passing a variable by reference with a pointer does not actually pass anything by reference.
  - Instead, a pointer to that variable is passed by value.
    - That pointer's value is copied into the function's corresponding pointer parameter.
  - The called function can then dereference the pointer parameter to access the caller's variable, thus accomplishing pass-by-reference.

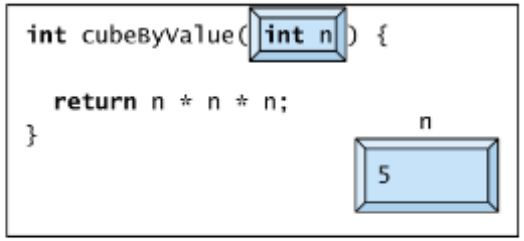
Graphical Analysis of Pass-By-Value and Pass-By-Reference

Step 1: Before `main` calls `cubeByValue`:

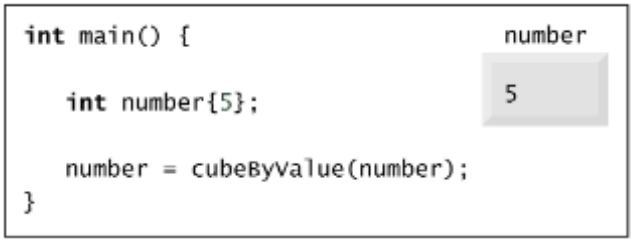


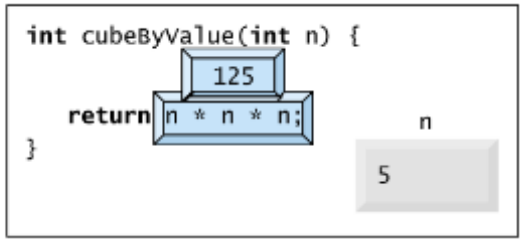
Step 2: After `cubeByValue` receives the call:





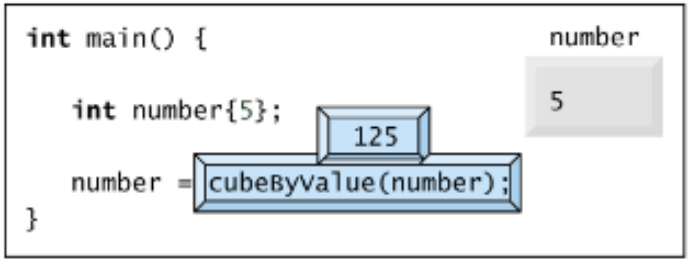
Step 3: After `cubeByValue` cubes parameter `n` and before `cubeByValue` returns to `main`:



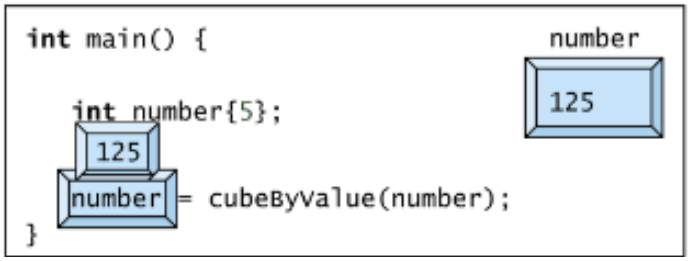


Graphical Analysis of Pass-By-Value and Pass-By-Reference

Step 4: After `cubeByValue` returns to `main` and before assigning the result to `number`:

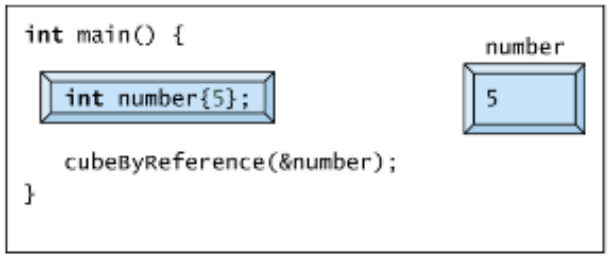


Step 5: After `main` completes the assignment to `number`:

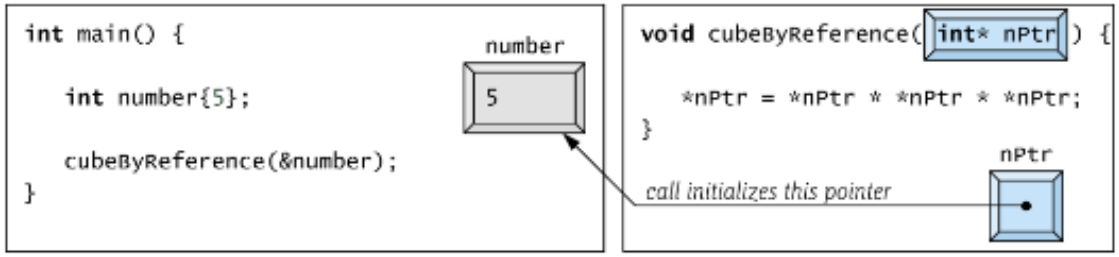


Graphical Analysis of Pass-By-Value and Pass-By-Reference

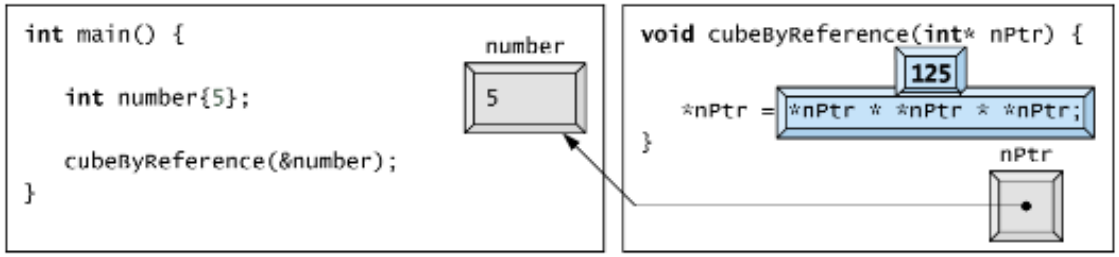
Step 1: Before `main` calls `cubeByReference`:



Step 2: After `cubeByReference` receives the call and before `*nPtr` is cubed:

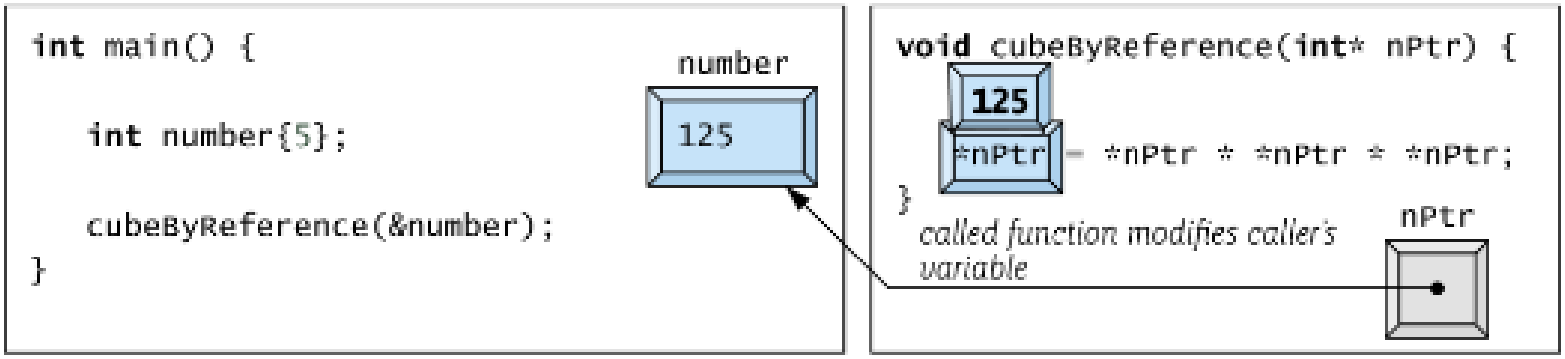


Step 3: Before `*nPtr` is assigned the result of the calculation `5 * 5 * 5`:

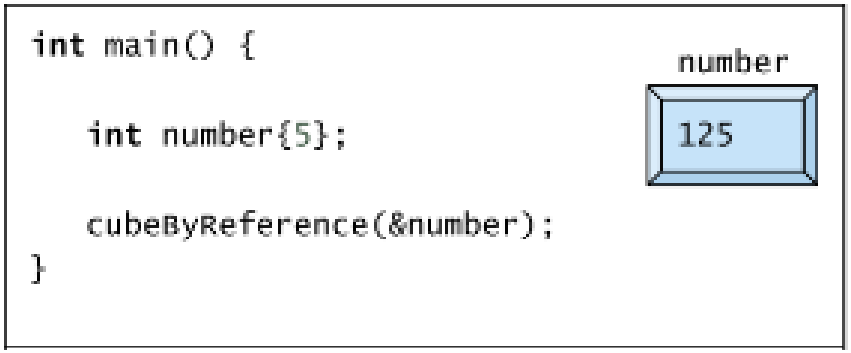


Graphical Analysis of Pass-By-Value and Pass-By-Reference.

Step 4: After `*nPtr` is assigned `125` and before program control returns to `main`:



Step 5: After `cubeByReference` returns to `main`:



Built-In Arrays

- Here we present built-in arrays, which like std::arrays, are fixed-size data structures.
  - We include this presentation mostly because you'll see built-in arrays in legacy C++ code.
  - New applications generally should use std::array and std::vector to create safer, more robust applications.
- In particular, std::array and std::vector objects always know their own size— even when passed to other functions, which is not the case for built-in arrays.

Built-In Arrays (Cont.)

- If you work on applications containing built-in arrays, you can
  - use C++20's to_array function to convert them to std::arrays, or
  - process them more safely using C++20's spans.
- Sometimes built-in arrays are required, such as when processing command-line arguments.

Built-In Arrays (Cont.)

- Declaring and Accessing a Built-In Array
  - As with std::array, you must specify a built-in array's element type and number of elements, but the syntax is different. For example, to reserve five elements for a built-in array of ints named c, use
    int c[5]; // c is a built-in array of 5 integers
  - You use the subscript ([]) operator to access a built-in array's elements.
    - Recall from Chapter 6 that the subscript ([]) operator does not provide bounds checking for std::arrays—this is also true for built-in arrays. Of course, std::array's at member function does bounds checking.
    - Built-in arrays do not have a range-checked at indexing function, but you can use the Guidelines Support Library (GSL) function gsl::at, which receives as arguments the array and an index.This function's first argument must be the array's name, not a pointer to the array's first element.

Built-In Arrays (Cont.)

- Initializing Built-In Arrays
  - You can initialize the elements of a built-in array using an initializer list. For example,
    int n[5]{50, 20, 30, 10, 40};
    creates and initializes a built-in array of five ints.
  - If you provide fewer initializers than the number of elements, the remaining elements are value initialized by default:
    - fundamental numeric types are set to 0,
    - bools are set to false,
    - pointers are set to nullptr, and
    - objects receive the default initialization specified by their class definitions.
  - If you provide too many initializers, a compilation error occurs.
  - The compiler can size a built-in array by counting its initializer list's elements. For example, the following creates a five-element array:
    int n[]{50, 20, 30, 10, 40};

Built-In Arrays (Cont.)

- Passing Built-In Arrays to Functions
  - The value of a built-in array's name is implicitly convertible to a pointer to the array's first element. This is known as decaying to a pointer.
  - Given the array
    int n[]{50, 20, 30, 10, 40};
    the name n is implicitly convertible to &n[0], which is a pointer to the element containing 50.
    - You don't need to take a built-in array's address (&) to pass it to a function—just pass its name.
    - As you saw in Section 7.4, a function that receives a pointer to a variable in the caller can modify that variable in the caller.
    - For built-in arrays, the called function can modify all of the array's elements in the caller—unless the parameter is declared **const** to prevent the argument array in the caller from being modified.

Built-In Arrays (Cont.)

- Declaring Built-In Array Parameters
  - You can declare a built-in array parameter in a function header as follows:
    int sumElements(const int values[], size_t numberOfElements)
  - Here, the function's first argument should be a one-dimensional built-in array of const ints.
  - Unlike std::arrays and std::vectors, built-in arrays don't know their own size, so a function that processes a built-in array also should receive the built-in array's size.

Built-In Arrays (Cont.)

- Declaring Built-In Array Parameters
  - The preceding header also can be written as
    int sumElements(const int* values, size_t numberOfElements)
  - **The compiler does not differentiate between a function that receives a pointer and one that receives a built-in array.**
    - In fact, the compiler converts const int values[] to const int* values under the hood.
    - This means the function must “know” when it’s receiving a pointer to a built-in array’s first element vs. a single variable being passed by reference.

Built-In Arrays (Cont.)

- Declaring Built-In Array Parameters
  - The C++ Core Guidelines say NOT to pass built-in arrays to functions as just a pointer.
    - Instead, you should pass C++20 spans because they maintain a pointer to the array's first element and the array's size.
    - You'll see that passing a span is superior to passing a built-in array and its size to a function.

Built-In Arrays (Cont.)

- Standard Library Functions **begin** and **end**
  - In Chapter 6, we sorted a std::array of strings called colors as follows:
    std::sort(std::begin(colors), std::end(colors));
  - Functions begin and end specified that the entire std::array should be sorted.
  - Function sort (and many other C++ standard library functions) also can be applied to built-in arrays. For example, to sort the built-in array n, you can write
    std::sort(std::begin(n), std::end(n)); // sort built-in array n
    - For a built-in array, begin and end work only with the **array's name**, not with a pointer to the array's first element.
    - Again, you should pass built-in arrays to other functions using C++20 spans.

Built-In Arrays (Cont.)

- Built-In Array Limitations
  - They cannot be compared using the relational and equality operators.
    - You must use a loop to compare two built-in arrays element by element.
    - If you had two int arrays named array1 and array2, the condition array1 == array2 would always be false, even if the arrays' contents are identical.
    - Remember, array names decay to const pointers to the arrays' first elements.
    - And, of course, for separate arrays, those elements reside at different memory locations.

Built-In Arrays.

- Built-In Array Limitations
  - Their elements cannot be assigned all at once other built-in arrays with simple assignments, as in builtInArray1 = builtInArray2.
  - They do not know their own size, so a function that processes a built-in array typically requires both the built-in array's name and size as arguments.
  - They don't provide automatic bounds checking.
    - You must ensure that array-access expressions use subscripts within the built-in array's bounds.

Using C++20 to_array to Convert a Built-in Array to a std::array

- In industry, you'll encounter C++ legacy code containing built-in arrays.
- The C++ Core Guidelines say you should prefer std::arrays and std::vectors to built-in arrays because they're safer and do not decay to pointers when you pass them to functions.
- C++20's std::to_array function (header <array>) conveniently creates a std::array from a built-in array or an initializer list.

Using C++20 to_array to Convert a Built-in Array to a std::array

- The next program demonstrates to_array.
  - We use a generic lambda expression (lines 8–14) to display each std::array's contents.
  - Again, specifying a lambda parameter's type as auto enables the compiler to infer the parameter's type based on the context in which the lambda appears.
  - In this program, the generic lambda automatically determines the element type of the std::array over which it iterates.

```
1   // C++20: Creating std::arrays with to_array.
2   #include <format>
3   #include <iostream>
4   #include <array>
5
6   int main() {
7      // generic lambda to display a collection of items
8      const auto display{
9         [](const auto& items) {
10           for (const auto& item : items) {
11              std::cout << std::format("{} ", item);
12           }
13        }
14     };
```

Using C++20 to_array to Convert a Built-in Array to a std::array (Cont.)

- Using to_array to Create an std::array from a Built-In Array
  - Line 19 creates a three-element std::array of ints by copying the contents of the built-in array values1.
  - We use auto to infer the std::array variable's type and size.
    - A compilation error occurs if we declare the array's type and size explicitly and it does not match to_array's return value.
    - We assign the result to the variable array1.
    - Lines 21 and 23 display the std::array's size and contents to confirm that it was created correctly.

```
16    const int values1[]{10, 20, 30};
17
18    // creating a std::array from a built-in array
19    const auto array1{std::to_array(values1)};
20
21    std::cout << std::format("array1.size() = {}\n", array1.size())
22       << "array1: ";
23    display(array1); // use lambda to display contents
```

```
array1.size() = 3
array1: 10 20 30
```

Using C++20 to_array to Convert a Built-in Array to a std::array.

- Using to_array to Create an std::array from an Initializer List
  - Line 26 shows that to_array can create a std::array from an initializer list.
  - Lines 27 and 29 display the array's size and contents to confirm that it was created correctly.

```
25      // creating a std::array from an initializer list
26      const auto array2{std::to_array({1, 2, 3, 4})};
27      std::cout << std::format("\n\narray2.size() = {}\n", array2.size())
28         << "array2: ";
29      display(array2); // use lambda to display contents
30
31      std::cout << '\n';
32   }
```

```
array2.size() = 4
array2: 1 2 3 4
```

Using const with Pointers and the Data Pointed To

- If a value does not (or should not) change in the body of a function to which it's passed, declare the parameter const.
- Before using a function, check its function prototype to determine the parameters it can and cannot modify.
- There are four ways to declare pointers:
  - a nonconstant pointer to nonconstant data,
  - a nonconstant pointer to constant data,
  - a constant pointer to nonconstant data and
  - a constant pointer to constant data.
- Each combination provides a different level of access privilege.

Using a Nonconstant Pointer to Nonconstant Data

- The highest level of data access is granted by a non-constant pointer to non-constant data.
- In this case, the data can be modified through the dereferenced pointer, and the pointer can be modified to point to other data items.
- A declaration for a non-constant pointer to non-constant data does not include const.
- Such a pointer might be used to receive a string as an argument to a function that processes (and possibly modifies) each character in the string.

Using a Nonconstant Pointer to Constant Data

- A nonconstant pointer to constant data is
  - a pointer that can be modified to point to any data of the appropriate type, but
  - the data to which it points cannot be modified through that pointer.
- The declaration for such a pointer places const to the left of the pointer's type, as in
  const int* countPtr;
  - The declaration is read from right to left as "countPtr is a pointer to an integer constant" or,
  - more precisely, "countPtr is a nonconstant pointer to an integer constant."

Using a Nonconstant Pointer to Constant Data.

- Use pass-by-value to pass fundamental-type arguments (e.g., ints, doubles, etc.) unless the called function must directly modify the value in the caller.
  - This is another example of the principle of least privilege.
- If large objects do not need to be modified by a called function, pass them using references or pointers to constant data—though references are preferred.
  - This gives the performance benefits of pass-by-reference and avoids the copy overhead of pass-by-value.
  - Passing large objects using references to constant data or pointers to constant data also offers the security of pass-by-value.

Using a Constant Pointer to Nonconstant Data

- A constant pointer to nonconstant data is a pointer that
  - always points to the same memory location and
  - the data at that location can be modified through the pointer.
- The declaration for such a pointer places const to the left of the pointer's name, as in
  int* const ptr{&x};
  - The declaration is read from right to left as "ptr is a constant pointer to a nonconstant integer."
- Pointers that are declared const must be initialized when they're declared.
  - If the pointer is a function parameter, it's initialized with the pointer passed to the function.
  - Each successive call to the function reinitializes that function parameter.

Using a Constant Pointer to Nonconstant Data.

- Attempting to modify a constant pointer causes compiler to issue an error message.
- The nonconstant value to which ptr points can be modified using the dereferenced ptr, even though ptr itself has been declared const.

Using a Non-Constant Pointer to Non-Constant Data

- The minimum access privileges are granted by a constant pointer to constant data:
  - such a pointer always points to the same memory location and
  - the data at that location cannot be modified via the pointer.
- The declaration for such a pointer places const to the left of the pointer's type and name, as in
  const int* const ptr{&x};
  - This declaration is read from right to left as "ptr is a constant pointer to an integer constant."
- Attempting to modify the data to which ptr points and attempting to modify the address stored in the pointer variable cause compiler to issue an error.

`sizeof` Operator

- The compile-time unary operator sizeof determines the size in bytes of a built-in array, type, variable or constant during program compilation.
  - When applied to a built-in array's name, as in the next program (line 12), sizeof returns the total number of bytes in the built-in array as a value of type size_t.
  - The computer we used to compile this program stores double variables in 8 bytes of memory. Array numbers has 20 elements (line 9), so it uses 160 bytes in memory.
  - Applying sizeof to a pointer (line 20) returns the size of the pointer in bytes (8 on the system we used).

`sizeof` Operator (Cont.)

```
1   // Sizeof operator when used on a built-in array's name
2   // returns the number of bytes in the built-in array.
3   #include <format>
4   #include <iostream>
5
6   size_t getSize(double* ptr); // prototype
7
8   int main() {
9      double numbers[20]; // 20 doubles; occupies 160 bytes on our system
10
11     std::cout << std::format("Number of bytes in numbers is {}\n",
12        sizeof(numbers));
13
14     std::cout << std::format("Number of bytes returned by getSize is {}\n",
15        getSize(numbers));
16  }
17
18  // return size of ptr
19  size_t getSize(double* ptr) {
20     return sizeof(ptr);
21  }
```

```
Number of bytes in the array is 160
Number of bytes returned by getSize is 8
```

`sizeof` Operator (Cont.)

- Determining the Sizes of the Fundamental Types, a Built-In Array and a Pointer
  - The number of bytes used to store a particular data type may vary among systems and compilers.
    - When writing programs that depend on data type sizes, consider using the fixed-size integer types from header <cstdint>. For the complete list, see: https://en.cppreference.com/w/cpp/types/integer
  - Operator sizeof can be applied to any expression or type name.
    - When applied to a variable name or expression, the number of bytes used to store the corresponding type is returned.
    - The parentheses used with sizeof are required only if a type name (e.g., int) is supplied as its operand.
    - The parentheses used with sizeof are not required when sizeof's operand is an expression.
  - Remember that sizeof is a compile-time operator, so its operand will not be evaluated at runtime.

`sizeof` Operator.

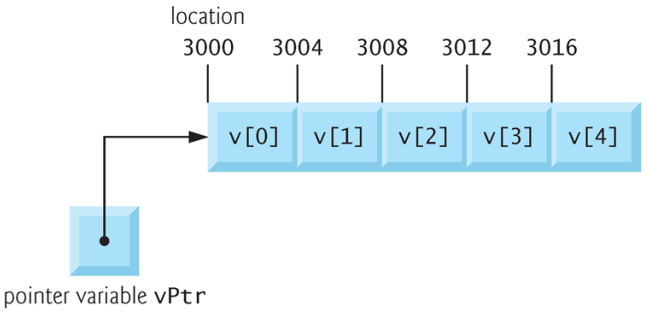
```cpp
// sizeof operator used to determine standard data type sizes.
#include <format>
#include <iostream>

int main() {
   constexpr char c{}; // variable of type char
   constexpr short s{}; // variable of type short
   constexpr int i{}; // variable of type int
   constexpr long l{}; // variable of type long
   constexpr long long ll{}; // variable of type long long
   constexpr float f{}; // variable of type float
   constexpr double d{}; // variable of type double
   constexpr long double ld{}; // variable of type long double
   constexpr int array[20]{}; // built-in array of int
   const int* const ptr{array}; // variable of type int*

   std::cout << std::format("sizeof c = {}\tsizeof(char) = {}\n",
      sizeof c, sizeof(char));
   std::cout << std::format("sizeof s = {}\tsizeof(short) = {}\n",
      sizeof s, sizeof(short));
   std::cout << std::format("sizeof i = {}\tsizeof(int) = {}\n",
      sizeof i, sizeof(int));
   std::cout << std::format("sizeof l = {}\tsizeof(long) = {}\n",
      sizeof l, sizeof(long));
   std::cout << std::format("sizeof ll = {}\tsizeof(long long) = {}\n",
      sizeof ll, sizeof(long long));
   std::cout << std::format("sizeof f = {}\tsizeof(float) = {}\n",
      sizeof f, sizeof(float));
   std::cout << std::format("sizeof d = {}\tsizeof(double) = {}\n",
      sizeof d, sizeof(double));
   std::cout << std::format("sizeof ld = {}\tsizeof(long double) = {}\n",
      sizeof ld, sizeof(long double));
   std::cout << std::format("sizeof array = {}\n", sizeof array);
   std::cout << std::format("sizeof ptr = {}\n", sizeof ptr);
}
```

```
sizeof c = 1    sizeof(char) = 1
sizeof s = 2    sizeof(short) = 2
sizeof i = 4    sizeof(int) = 4
sizeof l = 8    sizeof(long) = 8
sizeof ll = 8   sizeof(long long) = 8
sizeof f = 4    sizeof(float) = 4
sizeof d = 8    sizeof(double) = 8
sizeof ld = 16  sizeof(long double) = 16
sizeof array = 80
sizeof ptr = 8
```

Pointer Expressions and Pointer Arithmetic

- Pointers are valid operands in arithmetic expressions, assignment expressions and comparison expressions.
- However, not all the operators normally used in these expressions are valid in conjunction with pointer variables.
- This section describes the operators that can have pointers as operands, and how these operators are used.
  - A limited set of arithmetic operations may be performed on pointers.
  - A pointer may be incremented (++) or decremented (--), an integer may be added to a pointer (+ or +=), an integer may be subtracted from a pointer (- or -=) and one pointer may be subtracted from another—this last operation is meaningful only when both pointers point to elements of the same array.

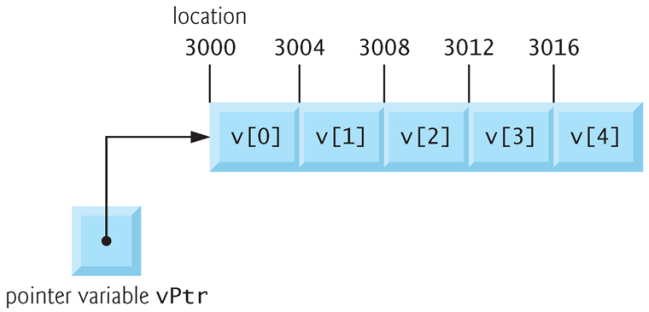
Pointer Expressions ar Arithmetic (Cont.)



- Assume that array `int v[5]` has been defined and its first element is at location 3000 in memory.
- Assume pointer `vPtr` has been initialized to point to `v[0]`—i.e., the value of `vPtr` is 3000.
- Variable `vPtr` can be initialized to point to array `v` with either of the statements:

```
int* vPtr{v};
int* vPtr{&v[0]};
```

Pointer Expressions ar Arithmetic (Cont.)



- Pointer Arithmetic
  - In conventional arithmetic, 3000 + 2 yields 3002.
    - This is normally not the case with pointer arithmetic.
  - When an integer is added to or subtracted from a pointer, the pointer is not incremented or decremented simply by that integer, but by that integer times the size of the object to which the pointer refers.
  - The number of bytes depends on the object's data type.
  - For example, the statement

    ```
    vPtr += 2;
    ```

    would produce 3008 (3000 + 2 * 4), assuming an integer is stored in 4 bytes of memory.
  - `vPtr` would now point to `v[2]`

Pointer Expressions and Pointer Arithmetic (Cont.)

- Pointer Arithmetic
  - If an integer is stored in 2 bytes of memory, then the preceding calculation would result in memory location 3004 (3000 + 2 * 2).
  - If the array were of a different data type, the preceding statement would increment the pointer by twice the number of bytes that it takes to store an object of that data type.
  - When performing pointer arithmetic on a character array, the results will be consistent with regular arithmetic, because each character is 1 byte long.

Pointer Expressions and Pointer Arithmetic (Cont.)

- Pointer Arithmetic
  - If `vPtr` had been incremented to 3016, which points to `v[4]`, the statement
    ```
    vPtr -= 4;
    ```
    would set `vPtr` back to 3000.
  - The increment (`++`) / decrement (`--`) operators can be used to increment / decrement a pointer.
    - Either of the statements
      ```
      ++vPtr;
      vPtr++;
      ```
      increments the pointer to point to the next location in the array.
    - Either of the statements
      ```
      --vPtr;
      vPtr--;
      ```
      decrements the pointer to point to the previous element of the array.

Pointer Expressions and Pointer Arithmetic (Cont.)

- Pointer Arithmetic.
  - There's no bounds checking on pointer arithmetic, so the C++ Core Guidelines recommend using std::spans instead, which we demonstrate later.
  - You must ensure that every pointer arithmetic operation that adds an integer to or subtracts an integer from a pointer results in a pointer that references an element within the built-in array's bounds.
  - As you'll see, std::spans have bounds checking, which helps you avoid errors.

Pointer Expressions and Pointer Arithmetic (Cont.)

- Subtracting One Pointer from Another
  - Pointer variables may be subtracted from one another.
    - For example, if `vPtr` contains the location 3000, and `v2Ptr` contains the address 3008, the statement

      ```
      x = v2Ptr - vPtr;
      ```

      would assign to x the number of array elements from `vPtr` to `v2Ptr`, in this case 2 (not 8).
    - Pointer arithmetic is undefined unless performed on an array.
    - We cannot assume that two variables of the same type are stored contiguously in memory unless they're adjacent elements of a built-in array.
    - Subtracting or comparing two pointers that do not refer to elements of the same built-in array is a logic error.

Pointer Expressions and Pointer Arithmetic (Cont.)

- Pointer Assignment
  - A pointer can be assigned to another pointer if both have the same type.
  - The exception to this rule is the pointer to `void` (i.e., `void *`), which is a generic pointer that can represent any pointer type.
    - All pointer types can be assigned a pointer to `void`, and a pointer to `void` can be assigned a pointer of any type.
    - However, a pointer of type void* cannot be assigned directly to a pointer of another type—the pointer of type void* must first be static_cast to the proper pointer type.
      - For example, to cast a void* named ptr to an int*, you'd use int* intPtr{static_cast<int*>(ptr)};

Pointer Expressions and Pointer Arithmetic (Cont.)

- Cannot Dereference a void* Pointer
  - A void* pointer cannot be dereferenced.
    - For example, the compiler “knows” that an int* points to four bytes of memory on a machine with four-byte integers. Dereferencing an int* creates an lvalue that is an alias for the int’s four bytes in memory.
    - A void*, however, contains a memory address for an unknown data type. You cannot use a void* in pointer arithmetic, nor can you dereference a void* because the compiler does not know the type or size of the data to which the pointer refers.
  - The allowed operations on void* pointers are:
    - comparing void* pointers with other pointers,
    - casting void* pointers to other pointer types and
    - converting other pointer types to void* pointers.
  - All other operations on void* pointers are compilation errors.

Pointer Expressions and Pointer Arithmetic.

- Comparing Pointers
  - Pointers can be compared using equality and relational operators, but such comparisons are meaningless unless the pointers point to elements of the same array.
    - Pointer comparisons compare the addresses stored in the pointers.
    - A comparison of two pointers pointing to elements in the same array could show, for example, that one pointer points to a higher-numbered element of the array than the other pointer does.
    - A common use of pointer equality comparison is determining whether a pointer has the value nullptr (i.e., a pointer to nothing).

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements

- We now continue our Objects-Natural approach by taking C++20 span objects for a spin.
- A span (header) enables programs to **view** contiguous elements of a container, such as a built-in array, a std::array or a std::vector
- A span is a “view” into a container.
  - It “sees” the container’s elements but does not have its own copy of those elements.

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Earlier, we discussed how C++ built-in arrays decay to pointers when passed to functions.
  - In particular, the function's parameter loses the size information provided when you declared the array.
  - You saw this in our sizeof demonstration.
- The C++ Core Guidelines recommend passing built-in arrays to functions as spans, which represent both a pointer to the array's first element and the array's size.
- The next program demonstrates some key span capabilities.

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Function displayArray
  - Passing a built-in array to a function typically requires the array's name and size.
  - Though the parameter items (line 11) is declared with [], it's simply a pointer to an int.
    - It does not "know" how many elements the function's argument contains.
  - There are various problems with this approach. For instance, the code that calls displayArray could pass the wrong value for size. In this case, the function
    - might not process all of items' elements, or
    - might access an element outside items' bounds—a logic error and a potential security issue.

```
 9   // items parameter is treated as a const int* so we also need the size to
10   // know how to iterate over items with counter-controlled iteration
11   void displayArray(const int items[], size_t size) {
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Function displayArray
  - In addition, we previously discussed the disadvantages of external iteration, as used in lines 12–14.
    - The C++ Core Guidelines checker in Visual Studio issues several warnings about displayArray and passing built-in arrays to functions.
    - We include function displayArray in this example only to compare it with passing spans in function displaySpan, which is the recommended approach.

```
1   // C++20 spans: Creating views into containers.
2   #include <array>
3   #include <format>
4   #include <iostream>
5   #include <numeric>
6   #include <span>
7   #include <vector>
8
9   // items parameter is treated as a const int* so we also need the size to
10  // know how to iterate over items with counter-controlled iteration
11  void displayArray(const int items[], size_t size) {
12     for (size_t i{0}; i < size; ++i) {
13        std::cout << std::format("{} ", items[i]);
14     }
15  }
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Function displaySpan
  - The C++ Core Guidelines indicate that a pointer should point only to one object, not an array.
  - They also indicate that functions like displayArray, which receive a pointer and a size, are error-prone.
  - To fix these issues, you should pass arrays to functions using spans.
  - Function displaySpan (lines 19–23) receives a span containing const ints—const because the function does not need to modify the data.

```
17   // span parameter contains both the location of the first item
18   // and the number of elements, so we can iterate using range-based for
19   void displaySpan(std::span<const int> items) {
20      for (const auto& item : items) { // spans are iterable
21         std::cout << std::format("{} ", item);
22      }
23   }
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Function displaySpan
  - A span encapsulates both a pointer and a size_t representing the number of elements.
  - When you pass a built-in array (or a std::array or std::vector) to displaySpan, C++ implicitly creates a span containing a pointer to the array's first element and its size, which the compiler determines from the array's declaration.
    - This span views the data in the original array you pass as an argument.
  - **The C++ Core Guidelines indicate that you can pass a span by value because it's just as efficient as passing the pointer and size separately**, as we did in displayArray.

```
17  // span parameter contains both the location of the first item
18  // and the number of elements, so we can iterate using range-based for
19  void displaySpan(std::span<const int> items) {
20     for (const auto& item : items) { // spans are iterable
21        std::cout << std::format("{} ", item);
22     }
23  }
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Function displaySpan
  - A span has many capabilities similar to arrays and vectors, such as iteration via the range-based for statement.
  - Because a span is created based on the array's original size as determined by the compiler, the range-based for guarantees that we cannot access an element outside the array's bounds.
    - This fixes the various problems associated with display-Array and helps prevent security issues like buffer overflows.

```
17  // span parameter contains both the location of the first item
18  // and the number of elements, so we can iterate using range-based for
19  void displaySpan(std::span<const int> items) {
20     for (const auto& item : items) { // spans are iterable
21        std::cout << std::format("{} ", item);
22     }
23  }
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Function times2
  - Because a span is a view into an existing container, changing the span's elements changes the container's original data.
  - Function times2 multiplies every item in its span<int> by 2.
  - Note that we use a non-const reference to modify each element that the span views.

```
25  // spans can be used to modify elements in the original data structure
26  void times2(std::span<int> items) {
27     for (int& item : items) {
28        item *= 2;
29     }
30  }
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Passing an Array to a Function to Display the Contents
  - Lines 33–35 create the int built-in array values1, the std::array values2 and the std::vector values3.
    - Each has five elements and stores its elements contiguously in memory.
  - Line 40 calls displayArray to display values1's contents.
    - The displayArray function's first parameter is a pointer to an int, so we cannot use a std::array's or std::vector's name to pass these objects to displayArray.

```
32  int main() {
33     int values1[]{1, 2, 3, 4, 5};
34     std::array values2{6, 7, 8, 9, 10};
35     std::vector values3{11, 12, 13, 14, 15};
36
37     // must specify size because the compiler treats displayArray's items
38     // parameter as a pointer to the first element of the argument
39     std::cout << "values1 via displayArray: ";
40     displayArray(values1, 5);
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Implicitly Creating spans and Passing Them to Functions
  - Line 45 calls displaySpan with values1 as an argument. The function's parameter was declared as
    std::span<const int>
    so C++ creates a span containing a const int* that points to the array's first element and a size_t representing the array's size, which the compiler gets from the values1 declaration (line 33).

```
33    int values1[]{1, 2, 3, 4, 5};
34    std::array values2{6, 7, 8, 9, 10};
35    std::vector values3{11, 12, 13, 14, 15};
```

```
42    // compiler knows values1's size and automatically creates a span
43    // representing &values1[0] and the array's length
44    std::cout << "\nvalues1 via displaySpan: ";
45    displaySpan(values1);
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Implicitly Creating spans and Passing Them to Functions
  - Because spans can view any contiguous sequence of elements, you may also pass a std::array or std::vector of ints to displaySpan (lines 49 and 51).
    - C++ will create an appropriate span representing a pointer to the container's first element and size.
    - This makes displaySpan more flexible than displayArray, which could receive only the built-in array in this example.

```
47    // compiler also can create spans from std::arrays and std::vectors
48    std::cout << "\nvalues2 via displaySpan: ";
49    displaySpan(values2);
50    std::cout << "\nvalues3 via displaySpan: ";
51    displaySpan(values3);
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Changing a span's Elements Modifies the Original Data
  - As we mentioned, function times2 multiplies its span's elements by 2.
  - Line 54 calls times2 with values1 as an argument. The function's parameter was declared as
    std::span<int>
    so C++ creates a span containing an int* that points to the array's first element and a size_t representing the array's size, which the compiler gets from the values1 declaration (line 33).

```
33    int values1[]{1, 2, 3, 4, 5};
34    std::array values2{6, 7, 8, 9, 10};
35    std::vector values3{11, 12, 13, 14, 15};

53    // changing a span's contents modifies the original data
54    times2(values1);
55    std::cout << "\n\nvalues1 after times2 modifies its span argument: ";
56    displaySpan(values1);
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Changing a span's Elements Modifies the Original Data
  - To prove that times2 modified the original array's data, line 56 displays values1' s updated values.
  - Like displaySpan, times2 can also be called with this program's std::array or std::vector.

```
53    // changing a span's contents modifies the original data
54    times2(values1);
55    std::cout << "\n\nvalues1 after times2 modifies its span argument: ";
56    displaySpan(values1);
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Manually Creating a Span and Interacting with It
  - You can explicitly create spans and interact with them.
  - Line 59 creates a span<int> that views the data in values1—the compiler uses CTAD to infer the element type int from values1's elements.
  - Lines 60–61 demonstrate the span's front and back member functions, which return the first and last elements of the view—thus, the first and last elements of values1, respectively.

```
58    // spans have various array-and-vector-like capabilities
59    std::span mySpan{values1}; // span<int>
60    std::cout << "\n\nmySpan's first element: " << mySpan.front()
61       << "\nmySpan's last element: " << mySpan.back();
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Manually Creating a Span and Interacting with It
  - A philosophy of the C++ Core Guidelines is to **"prefer compile-time checking to runtime checking."**
    - This enables the compiler to find and report errors at compile-time, rather than you writing code to help prevent runtime errors.
    - In line 59, the compiler determines the span's size (5) from the values1 declaration in line 33.
  - You can explicitly state the span's type and size, as in span<int, 5> mySpan{values1};
    - In this case, the compiler ensures that the span's declared size matches values1's size; otherwise, a compilation error occurs.

```
33    int values1[]{1, 2, 3, 4, 5};
34    std::array values2{6, 7, 8, 9, 10};
35    std::vector values3{11, 12, 13, 14, 15};

58    // spans have various array-and-vector-like capabilities
59    std::span mySpan{values1}; // span<int>
60    std::cout << "\n\nmySpan's first element: " << mySpan.front()
61       << "\nmySpan's last element: " << mySpan.back();
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Using a span with the Standard Library's accumulate Algorithm
  - As you've seen in this example, spans are iterable.
  - This means you also can use the begin and end functions with spans to pass them to C++ standard library algorithms, such as accumulate (line 65) or sort.
  - We cover standard library algorithms in depth in Chapter 14.

```
58    // spans have various array-and-vector-like capabilities
59    std::span mySpan{values1}; // span<int>
60    std::cout << "\n\nmySpan's first element: " << mySpan.front()
61       << "\nmySpan's last element: " << mySpan.back();
62
63    // spans can be used with standard library algorithms
64    std::cout << "\n\nSum of mySpan's elements: "
65       << std::accumulate(std::begin(mySpan), std::end(mySpan), 0);
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Creating Subviews
  - Sometimes, you might want to process subsets of a span.
  - A span's first, last and subspan member functions create subviews.
    - Lines 69 and 71 use **first** and **last** to get spans representing values1's first three and last three elements, respectively.
    - Line 73 uses **subspan** to get a span that views the 3 elements starting from index 1.
    - In each case, we pass the subview to displaySpan to confirm what the subview represents.

```
67    // spans can be used to create subviews of a container
68    std::cout << "\n\nFirst three elements of mySpan: ";
69    displaySpan(mySpan.first(3));
70    std::cout << "\nLast three elements of mySpan: ";
71    displaySpan(mySpan.last(3));
72    std::cout << "\nMiddle three elements of mySpan: ";
73    displaySpan(mySpan.subspan(1, 3));
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements (Cont.)

- Changing a Subview's Elements Modifies the Original Data
  - A subview of non-const data can modify that data.
  - Line 76 creates a span that views the 3 elements starting from index 1 of values1, then passes it to function times2.
  - Line 78 displays the updated values1 elements to confirm the results.

```
75    // changing a subview's contents modifies the original data
76    times2(mySpan.subspan(1, 3));
77    std::cout << "\n\nvalues1 after modifying elements via span: ";
78    displaySpan(values1);
```

Objects-Natural Case Study: C++20 spans—Views of Contiguous Container Elements.

- Accessing a View's Elements Via the [] Operator
  - Like built-in arrays, std::arrays and std::vectors, you can access and modify span elements via the [] operator, which does **NOT** provide range checking.
  - Line 81 displays the element at index 2.

```
80      // access a span element via []
81      std::cout << "\n\nThe element at index 2 is: " << mySpan[2];
82  }
```

A Brief Intro to Pointer-Based Strings

- We've already used the C++ standard library string class to represent strings as full-fledged objects.
  - Chapter 8 presents class std::string in detail.
- This section introduces pointer-based strings, as inherited from the C programming language.
- Here, we'll refer to these as C-strings or strings and use std::string when referring to the C++ standard library's string class.

A Brief Intro to Pointer-Based Strings (Cont.)

- std::string is preferred because it eliminates many security problems and bugs caused by manipulating C-strings.
- However, there are some cases where C-strings are required, such as when processing command-line arguments.
- Also, if you work with legacy C and C++ programs, you'll likely encounter pointer-based strings.

A Brief Intro to Pointer-Based Strings (Cont.)

- Characters and Character Constants
  - Characters are the fundamental building blocks of C++ source programs.
    - Every program is composed of characters that—when grouped meaningfully—are interpreted by the compiler as instructions and data used to accomplish a task.
    - A program may contain character constants, each of which is an integer value represented as a character in single quotes.
    - The value of a character constant is the integer value of the character in the machine's character set.
      - For example, 'z' represents the letter z's integer value (122 in the ASCII character set) as type char.
      - Similarly, '\n' represents the integer value of newline (10 in the ASCII character set).

A Brief Intro to Pointer-Based Strings (Cont.)

- Pointer-Based Strings
  - A C-string is a built-in array of characters ending with a null character ('\0'), which marks where the string terminates in memory.
  - A C-string is accessed via a **pointer** to its first character (no matter how long the string is).

A Brief Intro to Pointer-Based Strings (Cont.)

- String Literals as Initializers
  - A string literal may be used as an initializer for a built-in array of chars or a variable of type const char*.
  - The following declarations each initialize a variable to the string "blue":
    char color[]{"blue"};
    const char* colorPtr{"blue"};
    - The first declaration creates a five-element built-in array color containing the characters 'b', 'l', 'u', 'e' and '\0’.
    - The second creates the pointer variable colorPtr pointing to the letter b in the string "blue" (which ends in '\0') somewhere in memory.
  - The first declaration above also may be implemented using an initializer list of individual characters in which you manually include the terminating '\0', as in:
    char color[]{'b', 'l', 'u', 'e', '\0'};

A Brief Intro to Pointer-Based Strings (Cont.)

- String Literals as Initializers
  - String literals exist for the duration of the program.
  - They may be shared if the same string literal is referenced from multiple locations in a program.
  - String literals are immutable—they cannot be modified.

A Brief Intro to Pointer-Based Strings (Cont.)

- Problems with C-Strings
  - Not allocating sufficient space in a built-in array of chars to store the null character that terminates a string is a logic error.
  - Creating or using a C-string that does not contain a terminating null character can lead to logic errors.
  - When storing a string of characters in a built-in array of chars, be sure that it is large enough to hold the largest string that will be stored.
    - C++ allows strings of any length.
    - If a string is longer than the built-in array of chars in which it's to be stored, characters beyond the end of the built-in array will overwrite subsequent memory locations.
    - This could lead to logic errors, program crashes or security breaches.

A Brief Intro to Pointer-Based Strings (Cont.)

- Displaying C-Strings
  - A built-in array of chars representing a null-terminated string can be output with cout and <<.
  - The statement
    std::cout << color;
    displays the built-in array color.
  - The cout object does not care how large the built-in array of chars is.
    - The characters are output until a terminating null character is encountered.
    - The null character is not displayed.
  - Both cin and cout assume that built-in arrays of chars should be processed as strings terminated by null characters.
    - They do not provide similar input and output processing capabilities for other built-in array types.

A Brief Intro to Pointer-Based Strings (Cont.)

- Command-Line Arguments
  - There are cases in which built-in arrays and C-strings must be used.
  - For example, command-line arguments are often passed to applications to specify configuration options, file names to process and more.
  - You supply command-line arguments to a program by placing them **after** its name when executing it from the command line.

A Brief Intro to Pointer-Based Strings (Cont.)

- Command-Line Arguments
  - On a Windows system, the command
    dir /p
    uses the /p argument to list the contents of the current directory, pausing after each screen of information.
  - Similarly, on Linux or macOS, the following command uses the -la argument to list the contents of the current directory with details about each file and directory:
    ls -la

A Brief Intro to Pointer-Based Strings (Cont.)

- Command-Line Arguments
  - Command-line arguments are passed into a C++ program as C-strings.
    - The application name is treated as the first command-line argument.
  - To use the arguments as std::strings or other data types (int, double, etc.), you must convert the arguments to those types.
  - The next program displays the number of command-line arguments passed to the program, then displays each argument on a separate line.

A Brief Intro to Pointer-Based Strings (Cont.)

- Command-Line Arguments
  - To receive command-line arguments, declare main with two parameters (line 5), which by convention are named **argc** and **argv**, respectively.
    - The first is an int representing the number of arguments.

```
main Leila Haufiku 97
Number of arguments: 4
main
Leila
Haufiku
97
```

```
1   // Reading in command-line arguments.
2   #include <format>
3   #include <iostream>
4
5   int main(int argc, char* argv[]) {
6      std::cout << std::format("Number of arguments: {}\n\n", argc);
7
8      for (int i{0}; i < argc; ++i) {
9         std::cout << std::format("{}\n", argv[i]);
10     }
11  }
```

A Brief Intro to Pointer-Based Strings (Cont.)

- Command-Line Arguments
  - The second is a pointer to the first element of a built-in array of char*. Some programmers write this as char** argv.
    - The array's first element is a C-string for the application name.
    - The remaining elements are C-strings for the other command-line arguments.

```
main Leila Haufiku 97
Number of arguments: 4
main
Leila
Haufiku
97
```

```
1   // Reading in command-line arguments.
2   #include <format>
3   #include <iostream>
4
5   int main(int argc, char* argv[]) {
6       std::cout << std::format("Number of arguments: {}\n\n", argc);
7
8       for (int i{0}; i < argc; ++i) {
9           std::cout << std::format("{}\n", argv[i]);
10      }
11  }
```

A Brief Intro to Pointer-Based Strings (Cont.)

- Command-Line Arguments
  - The command
    main Leila Haufiku 97
    passes "Leila", "Haufiku" and 97" to the application main.
    - On macOS and Linux, you'd run this program with "./main".
  - Command-line arguments are separated by **whitespace**, not commas.
  - When this command executes, main's main function receives the argument count 4 and a four-element array of C-strings:
    - argv[0] contains the application's name "main" (or "./main" on macOS or Linux), and
    - argv[1] through argv[3] contain "Leila", "Haufiku" and "97", respectively.
  - You determine how to use these arguments in your program.
  - Due to the problems we've discussed with C-strings, you should convert the command-line arguments to std::strings before using them in your program.

A Brief Intro to Pointer-Based Strings (Cont.)

- Revisiting C++20's to_array Function
  - Previously we have demonstrated converting built-in arrays to std::arrays with to_array.
  - The next program shows another purpose of to_array.
  - We use the same lambda expression (lines 8–12) as in the previous example to display the std::array contents after the to_array call.

```
1   // C++20: Creating std::arrays from string literals with to_array.
2   #include <format>
3   #include <iostream>
4   #include <array>
5
6   int main() {
7      // lambda to display a collection of items
8      const auto display{
9         [](const auto& items) {
10           for (const auto& item : items) {
11              std::cout << std::format("{} ", item);
12           }
13        }
14     };
```

A Brief Intro to Pointer-Based Strings.

- Initializing a std::array from a String Literal Creates a One-Element array
  - Line 18 creates a one-element array containing a **const char*** pointing to the C-string "abc".

```
16
17  // initializing an array with a string literal
18  // creates a one-element array<const char*>
19  const auto array1{std::array{"abc"}};
20  std::cout << std::format("array1.size() = {}\narray1: ",
21     array1.size());
    display(array1); // use lambda to display contents
```

```
array1.size() = 1
array1: abc
```

A Brief Intro to Pointer-Based Strings.

- Passing a String Literal to to_array Creates a std::array of char
  - On the other hand, passing to_array a string literal (line 24) creates a **std::array** of chars containing elements for each character and the terminating null character.
    - Lines 25–26 confirm that the array's size is 6.
    - Line 27 confirms the array's contents.
    - The null character does not have a visual representation, so it does not appear in the output.

```
23    // creating std::array of characters from a string literal
24    const auto array2{std::to_array("C++20")};
25    std::cout << std::format("\n\narray2.size() = {}\narray2: ",
26       array2.size());
27    display(array2); // use lambda to display contents
28
29    std::cout << '\n';
30 }
```

```
array2.size() = 6
array2: C + + 2 0
```

Looking Ahead to Other Pointer Topics

- In later chapters, we'll introduce additional pointer topics:
  - In Chapter 10, OOP: Inheritance and Runtime Polymorphism, we'll use pointers with class objects to show that the "runtime polymorphic processing" associated with object-oriented programming can be performed with references or pointers—you should favor references.
  - In Chapter 11, Operator Overloading, Copy/Move Semantics and Smart Pointers, we introduce dynamic memory management with pointers, which allows you at execution-time to create and destroy objects as needed. Improperly managing this process is a source of subtle errors, such as "memory leaks." We'll show how "smart pointers" can automatically manage memory and other resources that should be returned to the operating system when they're no longer needed.

Looking Ahead to Other Pointer Topics.

- In Chapter 14, Standard Library Algorithms and C++20 Ranges & Views, we show that a function's name can be treated as a pointer to its implementation and that functions can be passed into other functions via function pointers.